

Worksheet No. - 8

Student Name: Ankush kumar

Branch: MCA

Semester: 2nd

Subject Name: TECHNICAL TRAINING

UID: 25MCA20305

Section/Group: 1A

Date of Performance: 1th Apr, 2026

Subject Code: 25CAP-652

Aim/Overview of the practical:

Apply stored procedure concepts for database operations

Objective:

To apply the concept of Stored Procedures in database operations in order to perform tasks like --

insertion, updating, deletion, and retrieval

of data efficiently, securely, and in a reusable manner within the database system.

Input/Apparatus Used:

- PostgreSQL
- pgAdmin

Procedure/Algorithm/Code :

```
SELECT * FROM EMPLOYEES;

--insertion

CREATE OR REPLACE PROCEDURE ADD_NEW_EMPLOYEE(IN EMP_ID INT, IN NAME
VARCHAR(50), IN DEPT_ID INT, IN SALARY INT )

LANGUAGE PLPGSQL

AS

$$

BEGIN

INSERT INTO EMPLOYEES (EMP_ID, NAME, DEPT_ID, SALARY)

VALUES (EMP_ID, NAME, DEPT_ID, SALARY);

END;
```

\$\$;

CALL ADD_NEW_EMPLOYEE(6, 'John', 30, 10000);

--updating

CREATE OR REPLACE PROCEDURE UPDATE_SALARY(IN IDS INT, INOUT BONUS INT, OUT
FINAL_SALARY INT)

LANGUAGE PLPGSQL

AS

\$\$

DECLARE

CURR_SALARY INT :=0;

BEGIN

SELECT salary INTO CURR_SALARY FROM EMPLOYEES WHERE EMP_ID = IDS;

-- Check if employee exists

IF NOT FOUND THEN

RAISE EXCEPTION 'Employee not found';

END IF;

BONUS := (BONUS::NUMERIC/100)*CURR_SALARY;

FINAL_SALARY := CURR_SALARY + BONUS;

UPDATE EMPLOYEES SET SALARY = FINAL_SALARY WHERE EMP_ID=IDS;

END;

\$\$;

call UPDATE_SALARY(1, 1, null);--first way of calling the procedure

--2nd way of calling the procedure

DO

\$\$

DECLARE

IDS INT :=1;

BONUS INT :=5;

FINAL_SALARY INT;

BEGIN

CALL UPDATE_SALARY(IDS,BONUS,FINAL_SALARY);

RAISE NOTICE 'HERE IS THE BONUS GIVEN TO THE EMPLOYEE %, AND THIS IS THE
FINAL SALRAY FOR HIM %',BONUS, FINAL_SALARY;

END;

\$\$;

--Deletion

CREATE OR REPLACE PROCEDURE delete_employee(

IN p_emp_id INT

)

LANGUAGE plpgsql

AS

\$\$

BEGIN

DELETE FROM employees

WHERE emp_id = p_emp_id;

IF NOT FOUND THEN

RAISE NOTICE 'Employee not found';

ELSE

RAISE NOTICE 'Employee deleted successfully';

END IF;

END;

\$\$;

CALL delete_employee(6);

--Retrival

CREATE OR REPLACE PROCEDURE get_employee(

IN p_emp_id INT,

OUT p_name VARCHAR,

OUT p_dept_id INT,

OUT p_salary INT

)

LANGUAGE plpgsql

AS

\$\$

BEGIN

SELECT name, dept_id, salary

INTO p_name, p_dept_id, p_salary

FROM employees

WHERE emp_id = p_emp_id;

IF NOT FOUND THEN

RAISE NOTICE 'Employee not found';

END IF;

END;

\$\$;

CALL get_employee(2, NULL, NULL, NULL);

Output:

	emp_id [PK] integer	name character varying (50)	dept_id integer	salary integer
1	2	Rohan	10	70000
2	3	Meera	20	40000
3	4	Kabir	20	55000
4	5	Isha	30	90000
5	1	Asha	10	55676

Figure 1: Initial Employees Table

```
NOTICE: Employee added successfully.
CALL

Query returned successfully in 63 msec.
```

Figure 2: Insertion using Stored Procedure

	bonus integer	final_salary integer
1	100	10100

Figure 3: Updating using direct call of stored procedure

```
NOTICE: Employee Updated Successfully.
NOTICE: HERE IS THE BONUS GIVEN TO THE EMPLOYEE 2784, AND THIS IS THE FINAL SALRAY FOR HIM 58460
DO

Query returned successfully in 85 msec.
```

Figure 4: Updating using call from do block

```
NOTICE: Employee deleted successfully  
CALL  
  
Query returned successfully in 65 msec.
```

Figure 5: Deletion of the employee using stored procedure

	p_name character varying	p_dept_id integer	p_salary integer
1	Rohan	10	70000

Figure 6: Retrieval using stored procedure

Learning outcomes (What I have learnt):

1. I learned how to create and use stored procedures to perform insertion, updating, deletion, and retrieval operations in a structured and reusable way.
2. I understood how to use different parameter types like IN, OUT, and INOUT to pass and return values within procedures.
3. I gained knowledge of implementing conditional checks such as `NOT FOUND` to handle cases where records may not exist.
4. I learned how to execute and test stored procedures using both direct `CALL` statements and anonymous blocks with `DO` for better control and output handling.